1 Experimental setup

To avoid repetition in the following answers, we summarize here the training parameters used across all experiments. In all cases, we used a learning rate of 0.001 and trained each network for 250 epochs, as in the original PointNet experiments. For Questions 1, 2, and 3, we used the ModelNet10 dataset, while in Question 4, we additionally used ModelNet40. All experiments were run on Google Colab using an NVIDIA T4 GPU.

2 Question 1

The best test accuracy achieved by the MLP model was 19.2% at epoch 19, while the evolution of both loss and accuracy over epochs is shown in Fig. 1. The full training took 271 seconds.

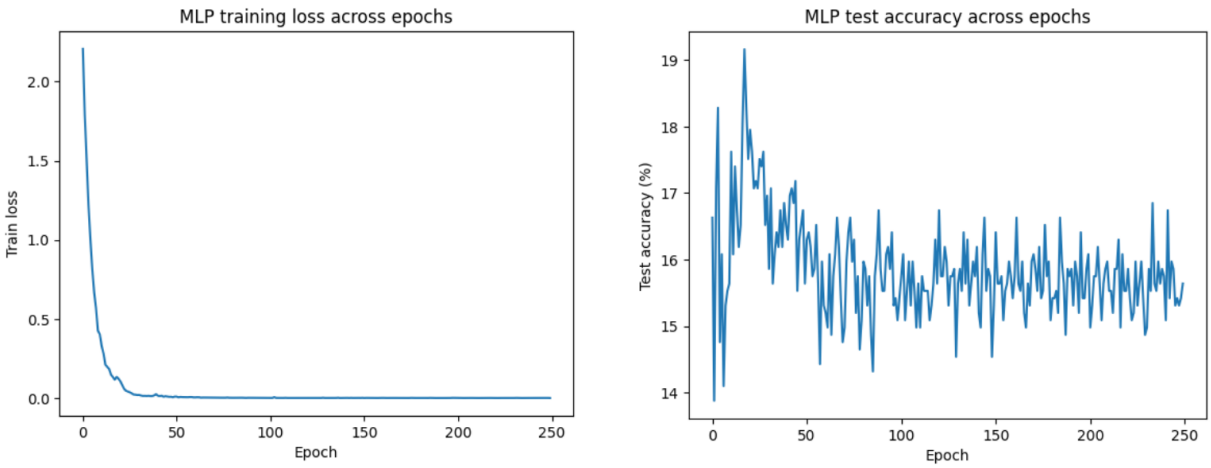


Figure 1: Training loss and test accuracy across epochs for the MLP model on ModelNet10.

From the plots, we observe that the training loss drops sharply and reaches near-zero values already around the 50th epoch. On the other hand, the test accuracy does not improve accordingly and instead oscillates mostly in the range of 15–17%. This behavior is a clear indication of overfitting.

The poor performance can also be attributed to the fact that the MLP flattens the input data, thereby discarding the spatial structure of the 3D point cloud. Moreover, points are tied to their index (in tensor representation), which makes the network sensitive to permutations of the input points. In general use case, this method also cannot handle variable input sizes, which is another limitation. Based on these observations, we conclude that a basic MLP is not well suited for 3D point cloud classification.

3 Question 2

The best test accuracy achieved by the basic PointNet model was 92.0% at epoch 111, while evolution of both loss and accuracy over epochs is shown in Fig. 2. The full training took 1143 seconds.

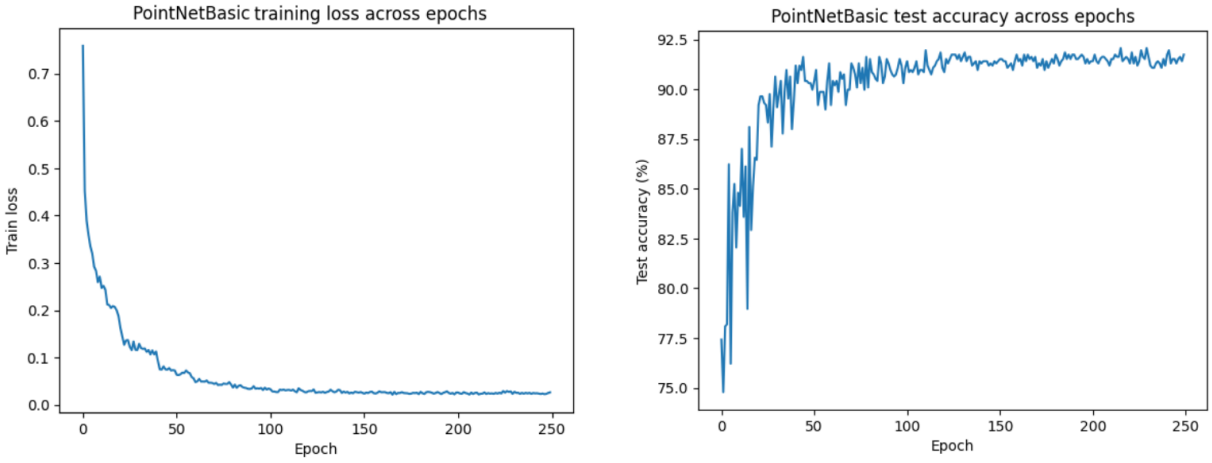


Figure 2: Training loss and test accuracy across epochs for the PointNetBasic model on ModelNet10.

From the plots, we observe that the training loss decreases steadily over epochs, while the test accuracy consistently improves and stabilizes at high values, unlike in the previous case. This confirms the out-performance of the PointNet model compared to the basic MLP.

The improvement can be attributed to the permutation invariance of PointNet, which is achieved through the symmetric aggregation operation (max pooling). Moreover, the network processes each point individually using shared MLP layers that lift the point features to higher dimensions, instead of flattening them into a single vector. These results are consistent with the motivation behind the design of PointNet and demonstrate why it is better suited for 3D point cloud classification.

4 Question 3

The best test accuracy achieved by the full PointNet model was 92.0% at epoch 97, while the evolution of both loss and accuracy over epochs is shown in Fig. 3. The full training took 2116 seconds.

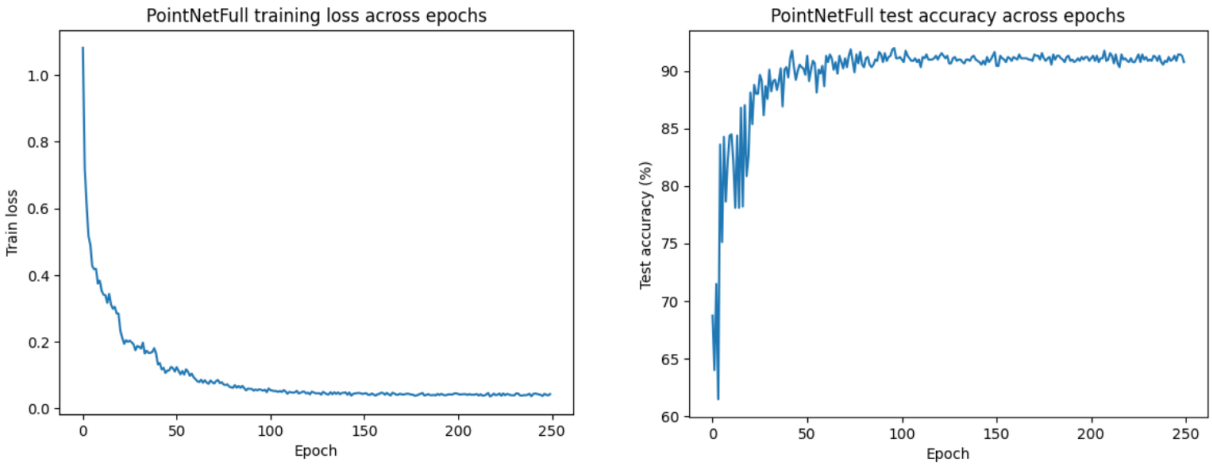


Figure 3: Training loss and test accuracy across epochs for the PointNetFull model on ModelNet10.

As in the previous experiment, the training loss decreases steadily and the test accuracy remains high and stable throughout training. Interestingly, the best test accuracy achieved with full PointNet is the same as with the basic PointNet model, although full PointNet reaches this best value at an earlier epoch. We also observe small differences in the plateau region, with full PointNet stabilizing at slightly (0-0.5%) lower accuracy values. Overall, however, these differences are minor and do not translate into a meaningful improvement in final performance, despite the expectation that the T-Net module would further improve accuracy.

A plausible explanation for the similar results is that the main purpose of the introduced T-Net is to learn a spatial transformation that aligns the input point cloud, and therefore increases the robustness to pose

variations. However, our training pipeline already includes random rotations around the z-axis as data augmentation, which forces the network to learn rotation-robust representations even without explicit alignment. As a consequence, the additional alignment provided by the T-Net offers small-to-none benefit in this setting.

A more noticeable difference, however, is the computational cost, with the full PointNet requiring approximately twice the training time and containing twice as many parameters (1,614,995 vs. 811,914) due to the additional T-Net module. Given that the accuracy improvement is negligible in our experiments, we can conclude that the basic PointNet model combined with appropriate data augmentation provides a better trade-off between performance and efficiency.

5 Question 4

To include a new data augmentation technique, we followed the approach used in PointNet++[1] and introduced `'Random point dropout'`. This augmentation randomly removes points from the point cloud, and therefore simulates occlusions that can occur in real sensing systems such as LiDAR. The idea is to encourage the network to rely on the global shape of the object rather than on a small subset of individual points. In practice, this should make the model more robust to missing data, sensor noise, and non-uniform sampling. In our implementation, we set the maximum dropout ratio to 0.4 and replace the dropped points with the first point in the cloud (instead of removing/discarding points, which would change the sample's tensor shape).

In the following experiments, we used the basic PointNet model with two augmentation settings: (1) the default augmentations (random rotation around the z-axis and random Gaussian noise) and (2) the custom augmentations (default augmentations with `'Random point dropout'` added). On ModelNet10 dataset, adding point dropout resulted in only a minor improvement in test accuracy (approximately +0.2%). In order to better evaluate the impact of the augmentation on generalization, we performed additional experiments on ModelNet40, which is a more challenging classification task with more classes, where such improvements should be more evident.

The best test accuracy achieved by the model with default augmentations was 86.0% at epoch 129, while the model with custom augmentations reached accuracy of 86.9% at epoch 66. Fig. 4 and 5 show the evolution of loss and accuracy over epochs for the default and custom augmentation settings, respectively. In addition to achieving a higher best accuracy (+0.9%), the model with `'Random point dropout'` converges significantly faster, shows (a bit) reduced oscillations in test accuracy curve, and stabilizes at a slightly higher accuracy plateau ($\geq$+1%) compared to the previous model. These observations indeed confirm that the additional augmentation improved the generalization, and hence, the overall performance.

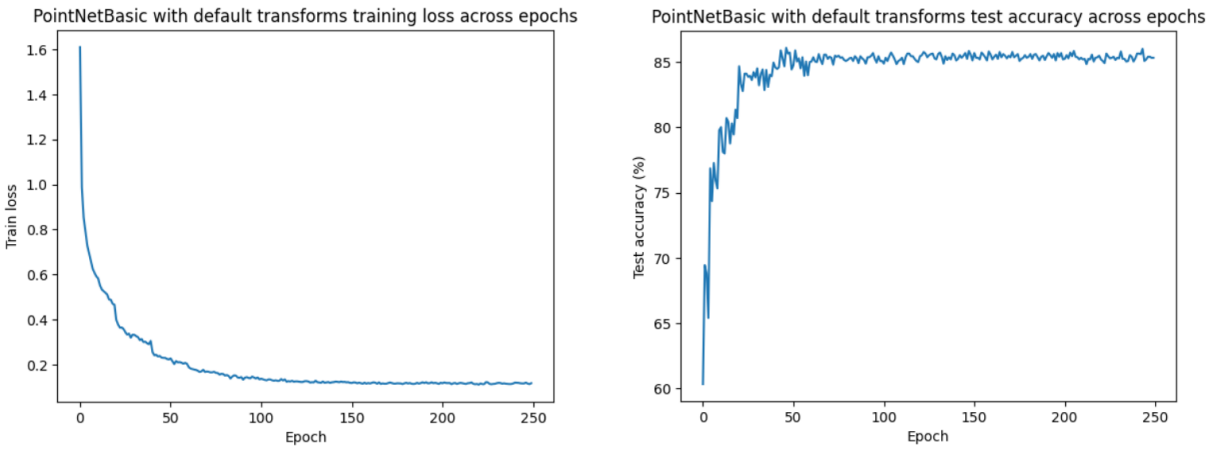


Figure 4: Training loss and test accuracy across epochs for the PointNetBasic model on ModelNet40 with default augmentations.

[1] https://deepwiki.com/yanx27/Pointnet_Pointnet2_pytorch/3.4-data-augmentation-techniques

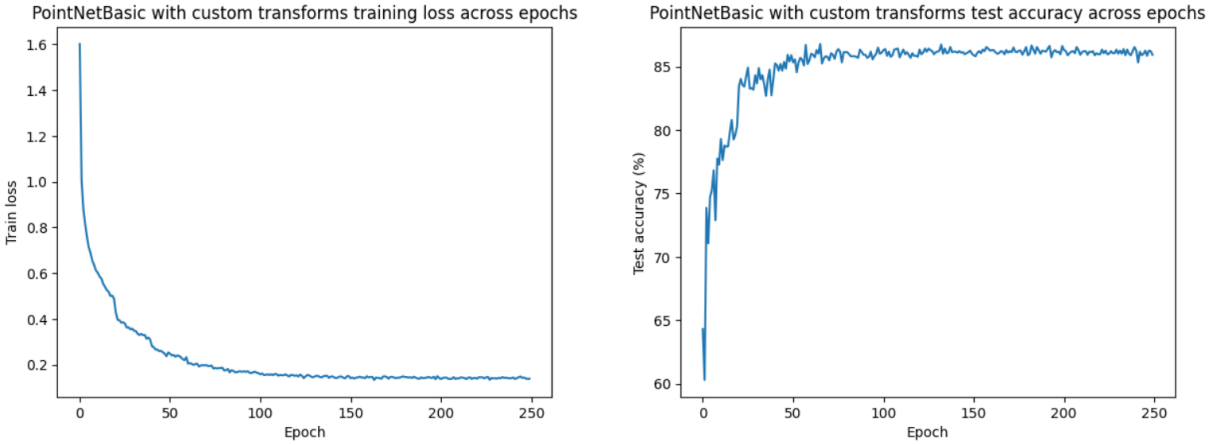


Figure 5: Training loss and test accuracy across epochs for the PointNetBasic model on ModelNet40 with custom augmentations.

As a final note, we compare our results with those reported in the PointNet paper, where an accuracy of 89.2% on ModelNet40 is achieved. Our best result (86.9%) approaches this benchmark, and we believe that with further refinements (e.g. learning rate or batch normalization decay), we would get the result even closer to the reported value.